



DEPARTMENT OF
COMPUTER SCIENCE



COMPUTATIONAL APPROACHES TO PROBLEM-SOLVING II

DR JEREMIAH O. BANDELE

PhD Electrical/Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- Describe more sophisticated types of computational approaches.



COMPUTATIONAL APPROACHES TO PROBLEM SOLVING

More sophisticated computational approaches to problem solving includes;

- ❖ Brute force
- ❖ Divide and conquer
- ❖ Greedy Algorithm
- ❖ Dynamic Programming
- ❖ Hypothesis Testing



BRUTE FORCE

With the brute force algorithm, a problem is solved by going through all possible choices until a solution is found.

This approach solely relies on computing power and works by trying every possibility instead of improving efficiency with advanced techniques.

Brute force algorithms are simple and consistent, but very slow.



BRUTE FORCE: EXAMPLE

Imagine that you have a small padlock with 4 digits, each from 0-9 and you forgot your combination.

How can you use a brute force method to open the lock?





DIVIDE AND CONQUER

With the divide and conquer strategy, a complex problem is first split into smaller easier parts.

After this split, each small easier part is solved; leading to a solution to the complex problem.





DIVIDE AND CONQUER

This approach can only work if the small easier parts are simpler than the complex part.

Solving the original problem directly is less complicated than combining the solutions of the small easier parts.



DIVIDE AND CONQUER: ANALOGY

Suppose a military commander wants to occupy a large territory. Occupying smaller territories at a time will help with the goal of occupying the large territory.





DIVIDE AND CONQUER: EXAMPLE

Let us consider the problem of sorting a set of numbers in ascending order using a divide and conquer algorithm called Merge sort.

5	2	4	6	1	3	2	6
---	---	---	---	---	---	---	---



1	2	2	3	4	5	6	6
---	---	---	---	---	---	---	---



GREEDY ALGORITHM

With the greedy algorithm, the best possible solution is applied at each stage when solving a problem. By always selecting the best optimal path, the greedy algorithm aims at achieving the optimal solution to a problem.



GREEDY ALGORITHM

The greedy algorithm might not always produce an optimal solution but they produce an optimal solution to many problems.

For problems where the greedy algorithm is able to provide optimal solutions, those solutions are always simple and highly efficient.



GREEDY ALGORITHM: EXAMPLE

Consider a set of numbers. 5, 9, 1, 4, 2 and 6.

If I asked you to tell me what the largest single digit number from the set of numbers is, what would be your answer?

If I then asked you to tell me the largest two digit number, how would you solve the problem in an efficient manner?



GREEDY ALGORITHM : EXAMPLE

You are given the start and finish times of six activities as start times of 2,4,1,6,9 and 6 and end times of 3,5,7,8,10 and 10 respectively.

With the assumption that you cannot work on more than one activity at a time, determine the maximum number of activities that you can perform?



DYNAMIC PROGRAMMING

With dynamic programming, problems are broken into sub-problems, and the solutions of the sub-problems are saved so that there is no need to compute the result again.



DYNAMIC PROGRAMMING

Provided that the solution to a problem exists, dynamic programming guarantees that an optimal solution can be found.

Dynamic programming is mostly used to solve optimization problems; where the task is to find out the minimum or the maximum solution of a problem.



DYNAMIC PROGRAMMING: EXAMPLE

Let us see an example of the Fibonacci series; a sequence in which each number is the sum of the two preceding ones as shown below;

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144



MID LESSON QUESTIONS

1. Consider a set of numbers, 2,8,4,9 and 3. If the number are to be sorted in descending order using the Merge sort algorithm. Show the step by step process the Merge sort algorithm goes through while solving the problem.



HYPOTHESIS TESTING

Hypotheses are tentative, intelligent guesses about the solution to the problem.

An hypothesis is a specific statement of prediction; it describes in concrete terms what you expect to happen.

An hypothesis is an assumption, that can be tested and proved to be right or wrong.

When dealing with hypotheses, it is common to define a null hypothesis. So what is a null hypothesis.



NULL HYPOTHESIS

Consider a problem to compare method A with method B in order to determine if both methods are good or not.

If the initial assumption is that both methods are equally good, then, this assumption is termed as the null hypothesis.

After stating the null hypothesis, the task is now about determining if the null hypothesis is true or not.



NULL HYPOTHESIS: SIGNIFICANCE LEVEL AND P VALUE

The significance level is the probability of rejecting the null hypothesis when it is true.

For instance, a significance level of 0.05 indicates a 5% risk of concluding that something exists when it does not exist.

The p-value (between 0 and 1) is a number that describes how likely it is that the null hypothesis is true
The smaller the p-value, the stronger the evidence that you should reject the null hypothesis



NULL HYPOTHESIS: ACCEPT/REJECT

When the p-value of the data is less than (or equal to) the significant level of the test, the null hypothesis is rejected.

However, for p-value higher than the significant value, the null hypothesis is accepted.

Note that rejecting the null hypothesis does not prove that something does not exist, it just means that the evidence is not enough.



NULL HYPOTHESIS: EXAMPLE

Hypothesis: Consumption of a particular medicine reduces the chances of heart arrest.

- ❖ **Null hypothesis:** The consumption of the medicine does not reduce the chances of heart arrest.



NULL HYPOTHESIS: EXAMPLE

Hypothesis: If random test scores are collected from men and women, does the score of one group differ from the other?”

- ◆ **Null hypothesis:** The mean test score of men is the same as that of the women.



FURTHER READING RESOURCES

- Levitin, A. (2008). Introduction To Design And Analysis Of Algorithms, 2/E. Pearson Education India.
- Lee, K. D., Lee, K. D., & Steve Hubbard, S. H. (2015). Data Structures and Algorithms with Python. Springer.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press.
- Shi, N., Tao, J. (2008). Statistical Hypothesis Testing: Theory and Methods. Singapore: World Scientific Publishing.



REFERENCES

- *Lee, K. D., Lee, K. D., & Steve Hubbard, S. H. (2015). Data Structures and Algorithms with Python. Springer.*



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU